

General Overview of the system

Topics

- History
- System Structure
- User Perspectives
- Operating System Services
- Assumptions About Hardware

1.1 History

- 1969
 - First UNIX at Bell Labs
 - The MULTICS
 - Kernighan, Ritchie, Thompson
- 1970's
 - Bell Labs makes UNIX freeware
 - Berkeley UNIX (BSD)
 - Bill Joy vi editor, C Shell
- 1980's
 - System V release 4
 - TCP/IP
 - Sun Microsystems Solaris
 - Microsoft Xenix, SCO
 - MIT X-Windows
- 1990's
 - GNU, LINUX
 - Stallman, Torvalds

Reasons for popularity and success of UNIX system

- System is written in a high level language
- Simple user interface
- Provides primitives that permit complex programs
- Uses a hierarchical file system
- Uses a consistent format for files
- Provides a simple, consistent interface to peripheral devices
- Multi user, multiprocess system
- Hides the machine architecture from the user

1.2 System Structure

- System Kernel
- System Calls
- Standard system configurations-Commands
- Low level programs
- High level view of the system

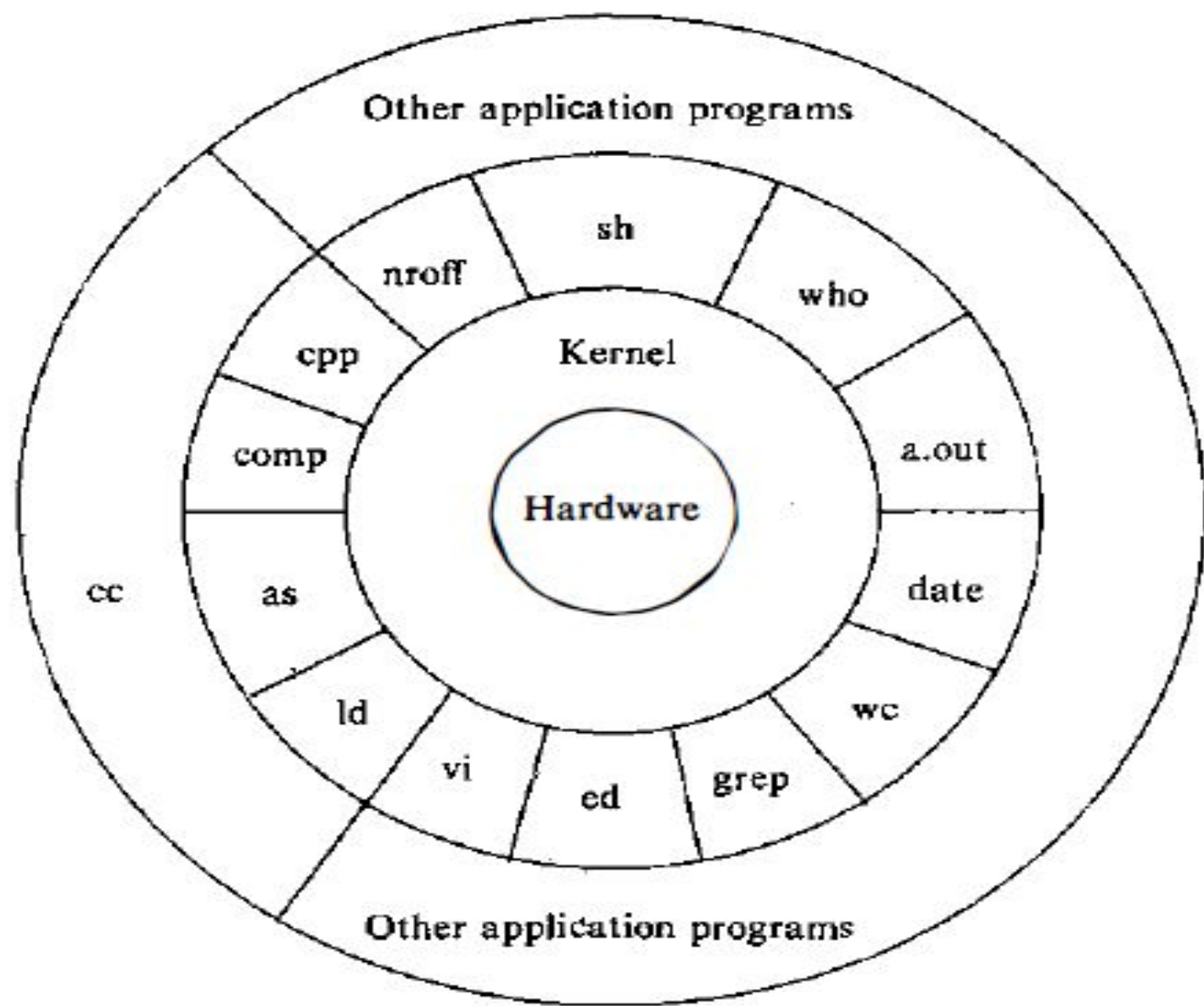


Figure 1.1. Architecture of UNIX Systems

1.3 User Perspectives

- **The File System**

The UNIX file system is characterized by

- A hierarchical structure
- Consistent treatment of file data
- The ability to create and delete files
- Dynamic growth of files
- The protection of file data
- The treatment of peripheral devices as files

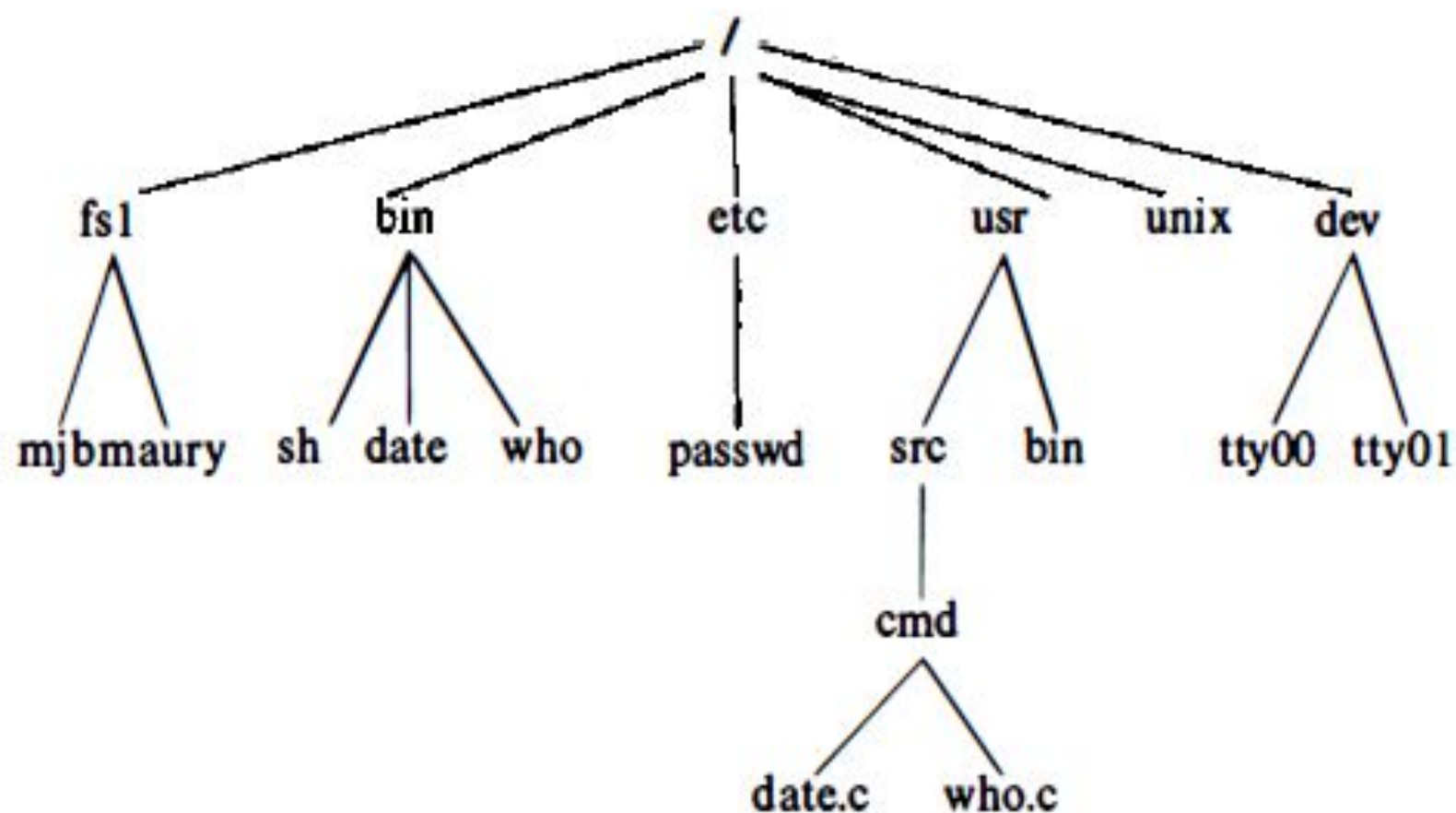


Figure 1.2. Sample File System Tree

```

#include <fcntl.h>
char buffer[2048];
int version = 1;      /* Chapter 2 explains this */

main(argc, argv)
    int argc;
    char *argv[];
{
    int fdold, fdnew;

    if (argc != 3)
    {
        printf("need 2 arguments for copy program\n");
        exit(1);
    }
    fdold = open(argv[1], O_RDONLY);  /* open source file read only */
    if (fdold == -1)
    {
        printf("cannot open file %s\n", argv[1]);
        exit(1);
    }
    fdnew = creat(argv[2], 0666);     /* create target file rw for all */
    if (fdnew == -1)
    {
        printf("cannot create file %s\n", argv[2]);
        exit(1);
    }
    copy(fdold, fdnew);
    exit(0);
}

copy(old, new)
    int old, new;
{
    int count;

    while ((count = read(old, buffer, sizeof(buffer))) > 0)
        write(new, buffer, count);
}

```

Figure 1.3. Program to Copy a File

Processing Environment

- program - executable file
- process - program executing
- multi-tasking; many processes executing where each process has it's own virtual machine
- system == collection of processes may be grouped by "job"; i.e., pipeline `wc | ls` and/or by user (all processes owned by jrb, group faculty)
- processes created by system calls; i.e., `fork()`, `exec()`,
overlay one process with code for another `wait()` - wait for process termination .
`exit()` - give up processor

```
main(argc, argv)
    int argc;
    char *argv[];
{
    /* assume 2 args: source file and target file */
    if (fork() == 0)
        execl("copy", "copy", argv[1], argv[2], 0);
    wait((int *) 0);
    printf("copy done\n");
}
```

Figure 1.4. Program that Creates a New Process to Copy Files

Building Block Primitives

- "pipes and filters"
- "to provide Operating system primitives that enable users to write small, modular programs that can serve as building bridges for more complex programs".
- pipe, shell can glue two processes together. Also an Operating system notion (it's a system call) that the shell uses. `cat list | sort | wc -l`

1.4 Operating System Services

Services provided by the kernel are:

- Controlling the execution of processes.
- Scheduling processes.
- Allocating main memory for an executing process.
- Allocating secondary memory for efficient storage and retrieval of user data.
- Allowing processes controlled access to peripheral devices such as terminals, tape drives, disk drives, & network devices.

1.5 Assumptions about Hardware

- process execution modes, USER/KERNEL (supervisor)
- kernel mode for privileged instructions (halt)
- user mode can access own text/data
- kernel mode can access kernel text/data easily
- hardware supports mode switching
- operating system tracks processes and switches between them

		Processes			
		A	B	C	D
Kernel Mode		K			K
User Mode			U	U	

Figure 1.5. Multiple Processes and Modes of Execution

Interrupts and Exceptions

- devices interrupt the system asynchronously.
- kernel saves process context, determines which interrupt occurred and "services" it.
- device priorities are set in hardware design/CPU/interrupt handling circuitry
- exceptions - various kind of CPU generated (not external) interrupts e.g., divide by zero.
- traps - program initiated interrupts

Processor Execution Levels (Priority Levels)

- Operating System will mask out certain interrupts during critical section handling

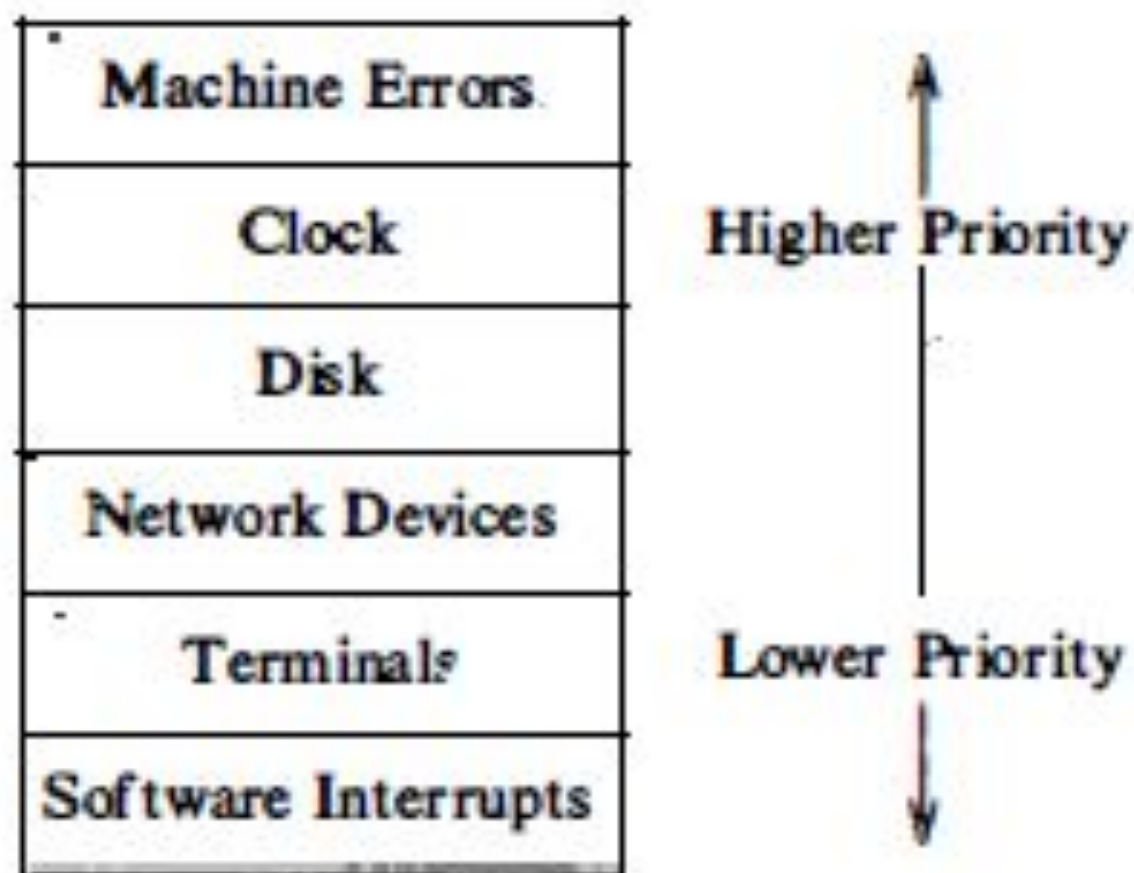


Figure 1.6 Typical Interrupt Levels

Memory Management

- kernel lives in low memory
- virtual machine == process; i.e., as if the only process in the system
- kernel works with mmu hardware to map virtual to physical addresses

Introduction to Kernel

- Architecture of the UNIX Operating System
- Introduction to System Concepts
- Kernel Data structures
- System Administration

Architecture of the UNIX Operating System

□ Three levels:

1. User
2. Kernel
3. Hardware

□ File subsystem

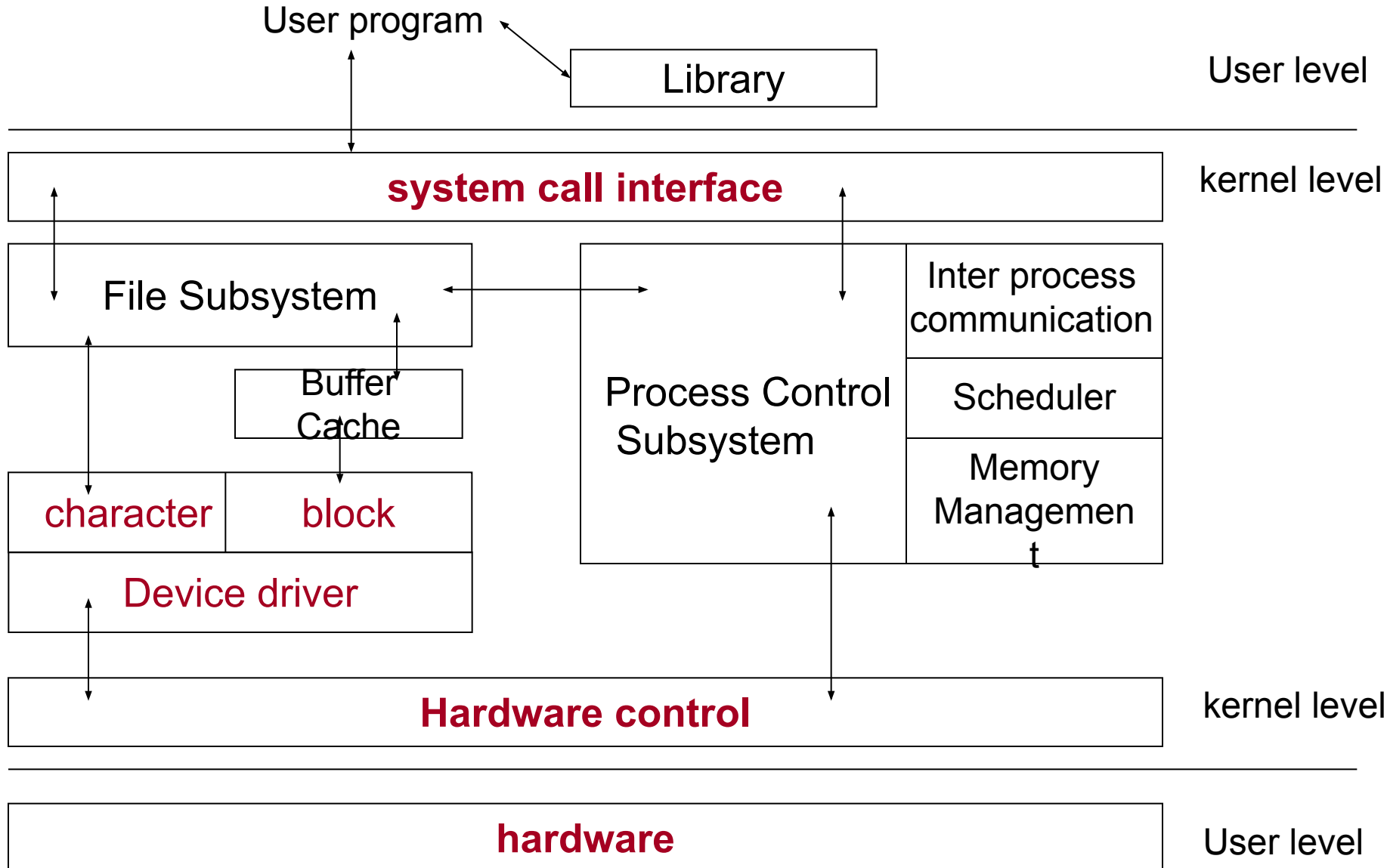
□ Process Control Subsystem

□ Memory Management Module

□ Scheduler module

□ Hardware control

Kernel Architecture (UNIX)



1.6 Introduction to System Concepts

It contain overview of some major kernel data structure used in a kernel architecture.

- An Overview of the File Subsystem
- Processes
 - Contexts of a process
 - Process states
 - State transitions
 - Sleep and wakeup

An Overview of the File Subsystem

- Internal representation of a file is given by an **inode**.
- Kernel contains two data structures
File table, user file descriptor table
- A file system consists of a sequence of logical blocks (512/1024 byte etc.)
- A file system has the following structure:

Boot Block	Super Block	Inode List	Data Blocks
------------	-------------	------------	-------------

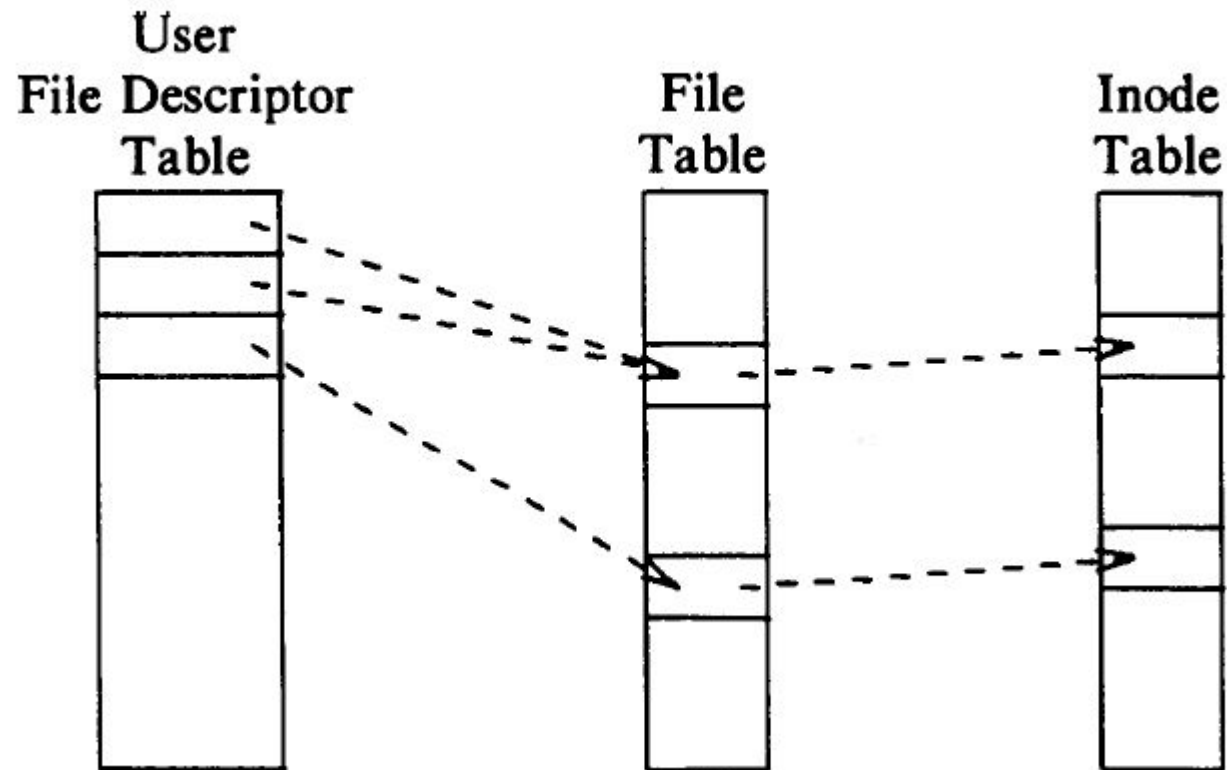


Figure: File Descriptors, File Table, and Inode Table

File System: Boot Block

- The beginning of the file system
- Contains bootstrap code to load the operating system
- Initialize the operating system
- Typically occupies the first sector of the disk

File System: Super Block

- Describes the state of a file system
- Describes the size of the file system
 - How many files it can store
- Where to find free space on the file system
- Other information

File System: Inode List

- Inodes are used to access disk files.
- Inodes maps the disk files
- For each file there is an inode entry in the inode list block
- Inode list also keeps track of directory structure

File System: Data Block

- Starts at the end of the inode list
- Contains disk files
- An allocated data block can belong to one and only one file in the file system

Processes

- A process is the execution of a program
- A process consists of **text** (machine code), **data** and **stack**
- Many processes can run simultaneously as kernel schedules them for execution
- Several processes may be instances of one program
- A process reads and writes its data and stack sections, but it cannot read or write the data and stack of other processes
- A process communicates with other processes and the rest of the world via **system calls**

Cont...

- Kernel has a **process table** that keeps track of all active processes
- Each entry in the process table contains pointers to the text, data, stack and the U Area of a process.
- All processes in UNIX system, except the very first process (process 0) which is created by the system boot code, are created by the **fork** system call

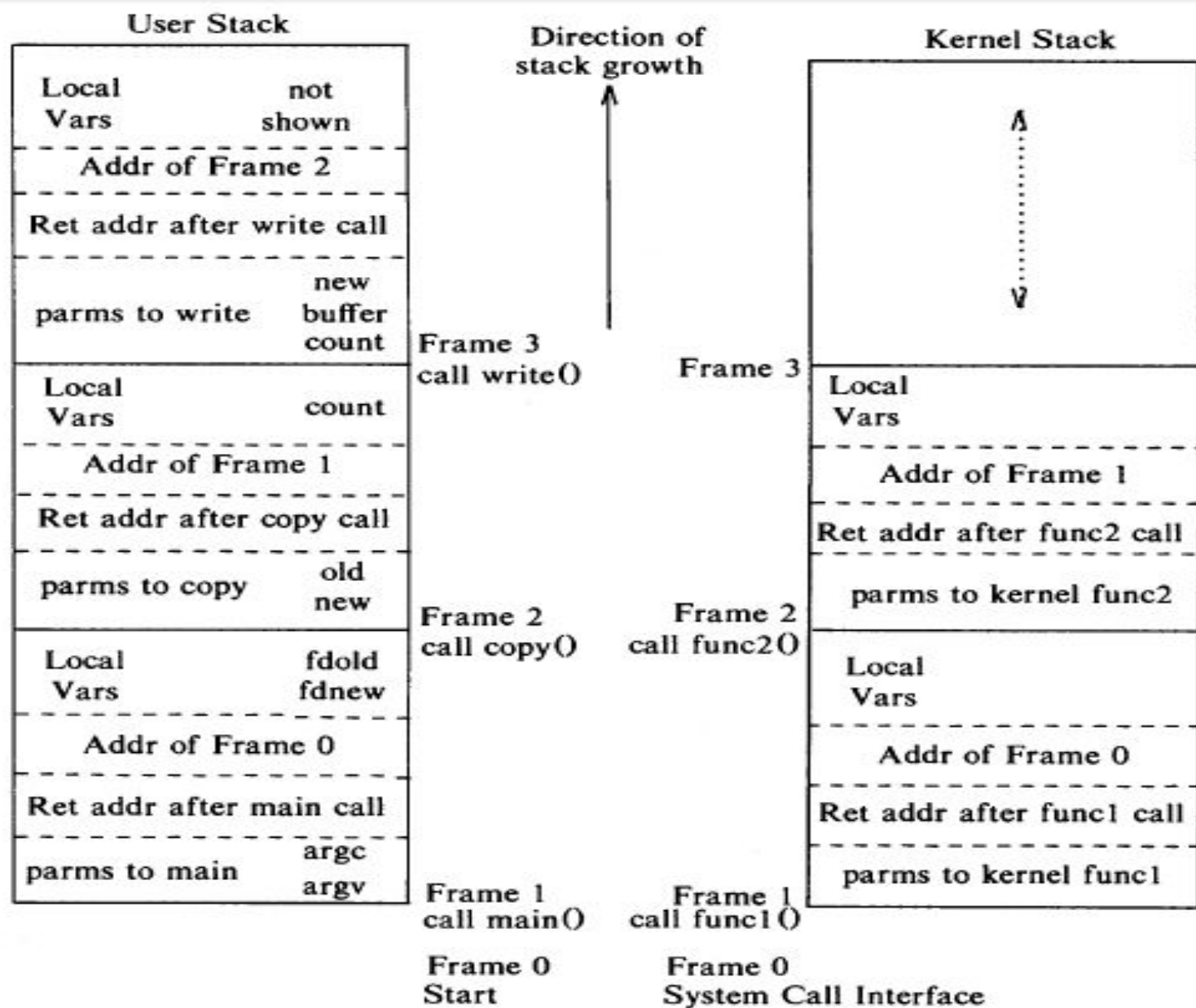


Figure: User and kernel stack for copy program

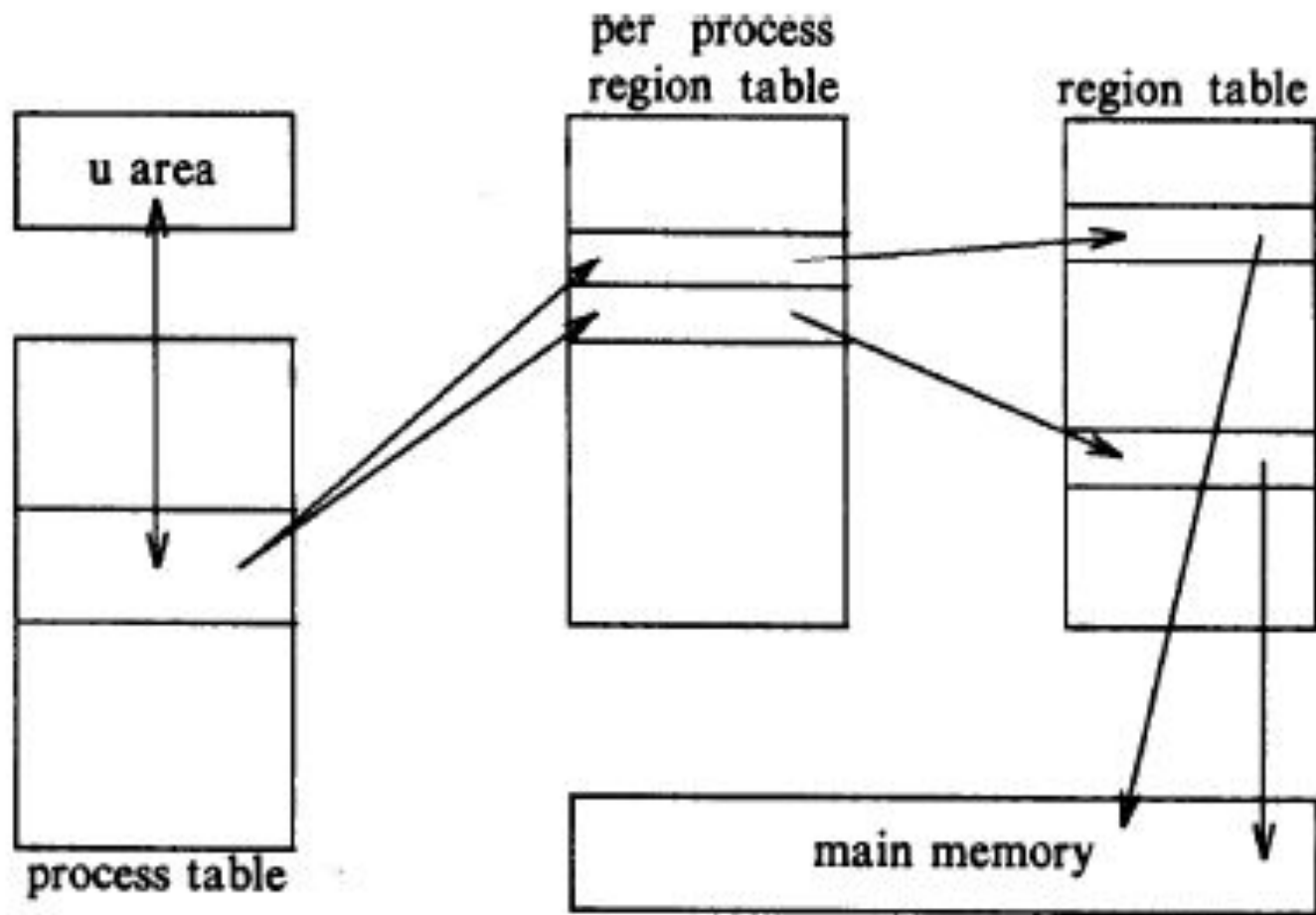


Figure: Data structures for Processes

Process: Region Table

- Region table entries describes the attributes of the region, such as whether it contains text or data, whether it is shared or private
- The extra level from the per process region table to kernel region table allows independent processes to **share** regions.

Process: U Area

- U Area is the extension of process table entry.
- Fields of process table entry:
 - State field
 - User ID (UID)
- Fields of U Area
 - Pointer to process table entry
 - File descriptors of all open files
 - Current directory and current root
 - I/O parameters
 - Process and file size limit
- Kernel can directly access fields of the U Area of the executing process but not of the U Area of other processes

Process Context

- The context of a process is its state:
 - Text, data(variable), register
 - Process region table, U Area,
 - User stack and kernel stack
- When executing a process, the system is said to be executing in the **context of the process**.

Context Switch

- When the kernel decides that it should execute another process, it does a context switch, so that the system executes in the context of the other process
- When doing a context switch, the kernel saves enough information so that it can later switch back to the first process and resume its execution.

Mode of Process Execution

- The UNIX process runs in two modes:
 - **User mode**
 - Can access its own instructions and data, but not kernel instruction and data
 - **Kernel mode**
 - Can access kernel and user instructions and data
- When a process executes a system call, the execution mode of the process changes from **user mode** to **kernel mode**

Mode of Process Execution

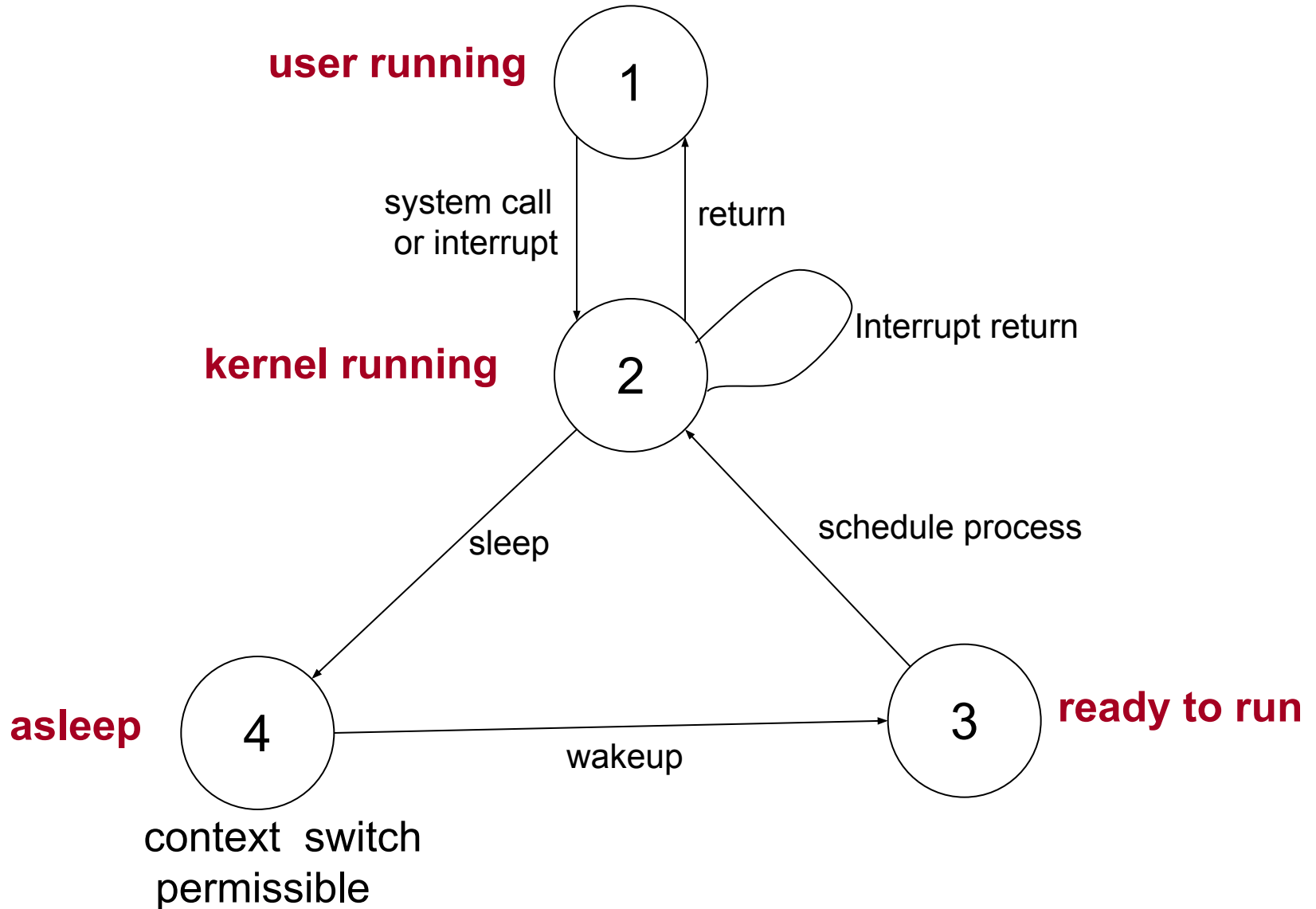
- When moving from user to kernel mode, the kernel saves enough information so that it can later return to user mode and continue execution from where it left off.
- Mode change is **not a context switch**, just change in mode.

Process States

Process states are:

- The process is running in user mode
- The process is running in kernel mode
- The process is not executing, but it is ready to run as soon as the scheduler chooses it
- The process is sleeping
 - Such as waiting for I/O to complete

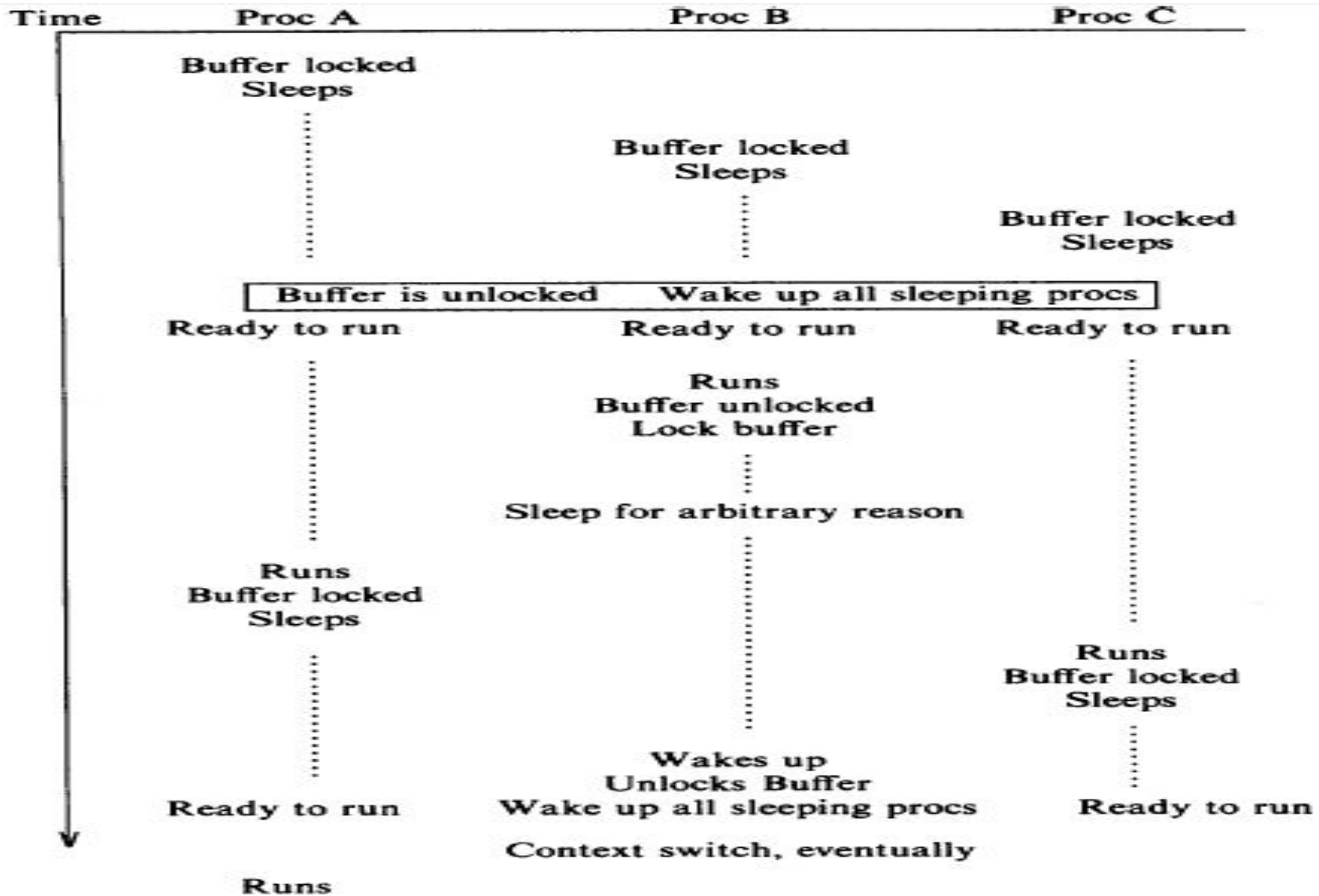
Process State Transition



Cont...

- The kernel allows a context switch only when a process moves from the state kernel running to the state asleep
- Process running in kernel mode cannot be preempted by other processes.

Sleep and Wakeup



Figure

Multiple Processes Sleeping on a Lock

1.7 Kernel Data Structures

- Most Kernel data structures occupy fixed size tables rather than dynamically allocated space.
- Advantage: Kernel code is simple

1.8 System Administration

- Administrative processes contains various functions:
 - .Disk Formatting
 - .Creation of new file systems
 - .Repair of damaged file systems
 - .Kernel debugging